

북극 해빙 면적 변화

11. CSV 파일을 불러오고 데이터의 첫 모습을 확인하기

의사코드 'seaice.csv'를 DataFrame 으로 불러온다 → 처음 5 행을 출력하여 열 이름과 데이터 형태를 확인한다.

확인하기 처음 5 행을 보면서 Year, Month, Day, Extent, hemisphere 등의 열이 있는지 확인합니다.

생각해 보기 IDLE 의 .py 파일에서는 `print(df.head())`로 출력

2. 열 이름의 불필요한 공백 제거하기

의사코드 열 이름에 앞뒤 공백이 있으면 열 선택에서 오류가 날 수 있다.

확인하기 열 이름의 앞뒤 공백이 제거됩니다.

생각해 보기 열 이름 오류가 보이면 데이터 값보다 먼저 `df.columns` 를 출력해 확인합니다.

3. 분석에 필요한 열만 선택하고 날짜 만들기

의사코드 Year, Month, Day, Extent, hemisphere 열만 남긴다 → Year·Month·Day 를 하나의 날짜(Date)로 변환한다

확인하기 Date 열이 새로 생성되어 연·월·일이 하나의 날짜 형식으로 표현됩니다.

생각해 보기 날짜를 만들어 두면 이후 월별·연도별 시계열 분석으로 확장하기 쉽습니다.

4. 전체 데이터를 바로 시각화하면 왜 해석하기 어려울까?

의사코드 Year 를 x 축, Extent 를 y 축으로 전체 데이터를 그려 본다

확인하기 같은 연도에 여러 관측값이 포함되어 있어 선이 겹치거나 복잡하게 연결될 수 있습니다.

핵심 질문 북극 해빙의 장기 변화 경향을 보려면 모든 지역·모든 달이 정말 필요한가?

5. 북극의 9 월 데이터만 선택하기

의사코드 hemisphere 가 'north'이고 Month 가 9 인 행만 남긴다 → Year 와 Extent 열만 선택한다.

확인하기 북극의 9 월 해빙 면적 분석에 필요한 Year 와 Extent 만 남습니다.

생각해 보기 9 월은 북극 해빙이 작아지는 시기의 변화를 살펴보기 위한 분석 조건입니다.

6. 결측치와 중복 데이터 확인하기

의사코드 각 열의 결측치 개수를 확인한다 → 완전히 같은 행이 중복되어 있는지 개수를 확인한다

확인하기 결측치나 중복이 있다면 연도별 평균을 계산하기 전에 먼저 처리해야 합니다.

생각해 보기 데이터 분석의 흐름은 확인 → 문제 발견 → 전처리 → 재분석입니다.

7. 같은 연도의 여러 값을 평균으로 묶기

의사코드 Year 가 같은 행들을 하나의 그룹으로 묶는다 → 각 연도의 Extent 평균을 계산한다 → index 를 일반 열로 되돌린다.

확인하기 한 연도당 하나의 평균 Extent 값이 만들어집니다.

생각해 보기 `groupby('Year')`는 같은 연도를 묶고, `['Extent'].mean()`은 각 그룹의 Extent 평균을 계산합니다.

8. 연도별 평균 해빙 면적 그래프 그리기

의사코드 연도를 x 축, 연도별 평균 Extent 를 y 축으로 선그래프를 그린다 → 전체 자료 그래프와 비교한다.

확인하기 연도별로 하나의 값만 사용하므로 장기적인 증가·감소 경향을 보다 쉽게 확인할 수 있습니다.

생각해 보기 분석 목적에 맞는 데이터가 만들어졌는지 확인하는 것이 중요합니다.

9. 직선 추세의 기울기와 절편 구하기

의사코드 numpy 를 불러온다 → Year 와 Extent 의 관계를 1 차식으로 맞춘다 → 기울기와 절편을 출력한다.

확인하기 기울기가 음수이면 시간이 지날수록 해빙 면적이 감소하는 방향의 추세를 뜻합니다. 기울기의 부호와 크기를 함께 해석합니다.

생각해 보기 np.polyfit(..., 1)의 1 은 자료에 1 차 직선 형태의 추세를 맞춘다는 뜻입니다.

10. 실제 변화선과 추세선을 함께 그리기

의사코드 기울기와 절편으로 각 연도의 추세값을 계산한다 → 실제 연도별 평균선을 그린다 → 같은 그래프에 점선 추세선을 추가한다 → 두 선을 비교한다.

확인하기 추세선은 전체 기간에서 장기적인 변화 방향을 요약해서 보여줍니다.

생각해 보기 추세선이 감소한다고 해서 모든 해의 값이 반드시 전년도보다 작아지는 것은 아닙니다. 실제 값의 변동과 장기 추세를 구분해서 해석합니다.

9 월 모의고사 데이터 분석

1. CSV 파일을 불러오고 데이터 구조 확인하기

의사코드 '9 월모의고사.csv'를 DataFrame 으로 불러온다 → 전체 데이터, 행·열의 크기, 처음 5 행을 확인한다 → 각 열의 자료형과 결측치 여부를 확인한다.

확인하기 전체 자료와 함께 (행의 수, 열의 수), 처음 5 행, 각 열의 자료형·결측치 정보가 순서대로 확인됩니다.

생각해 보기 IDLE 의 .py 파일에서는 결과를 확인하려면 print()가 필요한 경우가 많지만, info()는 자체적으로 내용을 출력합니다.

2. 한글 그래프를 위한 환경 설정과 첫 시각화

의사코드 Windows 에서 한글이 깨지지 않도록 글꼴을 설정한다 → 학생 이름을 x 축, 수학 등급을 y 축으로 막대그래프를 그린다 → x 축 글자를 회전하고 격자와 여백을 정리한 뒤 그래프를 출력한다.

확인하기 학생 이름과 수학 등급이 막대그래프로 나타나며 한글 제목과 축 글자가 정상적으로 표시됩니다.

생각해 보기 시각화는 데이터를 정리한 뒤 하는 것이 원칙이지만, 전처리 전 그래프를 먼저 그려 보면 데이터 문제를 발견하는 단서가 될 수 있습니다.

3. 결측치의 개수와 위치 확인하기

의사코드 각 열에 결측치가 몇 개 있는지 확인한다 → 수학 열에서 결측치가 있는 행의 index를 리스트 형태로 출력한다.

확인하기 어떤 열에 값이 비어 있는지와 수학 열의 결측치가 위치한 행 번호를 확인할 수 있습니다.

생각해 보기 결측치가 포함된 열은 정수형으로 바로 변환하려 할 때 오류가 발생할 수 있으므로 먼저 결측치를 확인해야 합니다.

4. 특정 결측 행을 제거하고 수학 등급을 정수로 바꾸기

의사코드 결측치가 있는 특정 index의 행을 제거한다 → 제거된 결과를 확인한다 → 수학 열을 정수형으로 변환하여 다시 출력한다.

확인하기 결측 행이 사라지고 수학 등급이 4.0, 2.0 형태가 아니라 4, 2 처럼 정수로 표시됩니다.

생각해 보기 drop(index=...)는 삭제할 행을 직접 지정하고, dropna()는 결측치가 있는 행을 자동으로 제거합니다.

5. 남은 결측치를 제거하고 등급 자료형 정리하기

의사코드 남아 있는 결측치가 포함된 행을 모두 제거한다 → 영어와 한국사 열을 정수형으로 변환한다
→ 전처리된 DataFrame 을 확인한다.

확인하기 결측치가 제거된 뒤 영어와 한국사 등급도 소수점 없이 정수로 표시됩니다.

생각해 보기 결측치 때문에 숫자 열이 실수형으로 읽힌 경우, 결측치를 처리한 다음 astype(int)로 자료형을 정리할 수 있습니다.

6. 국어·수학·영어 3 개 과목 등급 합 구하기

의사코드 국어, 수학, 영어 등급을 더한 새 열 '3 합'을 만든다 → 3 합이 8 이하인 행만 선택한다 → 이름과 세 과목 등급을 출력한다.

확인하기 세 과목 등급의 합이 기준을 만족하는 학생만 필터링되어 출력됩니다.

생각해 보기 등급은 숫자가 작을수록 좋은 결과이므로 조건을 만들 때 8 이상이 아니라 8 이하를 사용합니다.

7. 중복 데이터 확인하고 제거하기

의사코드 완전히 같은 행이 몇 개 중복되어 있는지 확인한다 → 중복으로 판단된 행의 index 를 확인한다 → 중복 행을 제거하고 결과를 출력한다.

확인하기 중복 개수와 뒤에 반복되어 나타난 행의 index 가 출력되고, 중복 제거 후에는 같은 데이터가 한 번만 남습니다.

생각해 보기 duplicated()는 기본적으로 처음 나타난 행은 남기고 이후에 다시 나타난 같은 행을 중복으로 판단합니다.

8. 전처리한 데이터로 수학 등급 선그래프 그리기

의사코드 그래프 크기를 정한다 → 학생 이름을 x 축, 수학 등급을 y 축으로 선그래프를 그린다 → 제목과 x 축,y 축 이름을 표시한다.

확인하기 전처리가 끝난 학생들의 수학 등급 변화가 이름 순서에 따라 선으로 연결되어 표시됩니다.

생각해 보기 중복 데이터가 남아 있으면 같은 이름 위치로 선이 다시 연결되어 그래프가 이상하게 보일 수 있으므로 전처리 후 시각화합니다.

9. 등급에 맞게 y 축 방향과 눈금 조정하기

의사코드 1 등급이 위쪽에 보이도록 y 축의 범위를 반대로 설정한다 → y 축에는 1 등급부터 9 등급까지 눈금을 표시한다 → 학생 이름이 겹치지 않도록 x 축 글자를 회전한다.

확인하기 그래프에서 1 등급은 위쪽, 9 등급은 아래쪽에 표시되고 모든 등급 눈금과 학생 이름을 쉽게 읽을 수 있습니다.

생각해 보기 ylim()은 보이는 축의 범위와 방향을 정하고, yticks()는 실제로 표시할 눈금 값을 지정합니다.

10. 최종 그래프를 보기 좋게 완성하기

의사코드 그래프에 격자를 추가한다 → 글자와 그래프 요소가 잘리지 않도록 여백을 정리한다 → IDLE 에서 별도의 그래프 창을 출력한다.

확인하기 격자와 여백이 정리된 최종 수학 등급 그래프가 별도 창에 나타납니다.

생각해 보기 xticks, yticks 는 눈금 표시를, xlim, ylim 은 축의 범위를 조절합니다. IDLE 에서는 마지막에 그래프 창을 직접 표시하는 명령이 필요합니다.

월별 전력사용량 변화

1. CSV 파일을 불러오고 데이터 구조 확인하기

의사코드 '전력사용량.csv'를 DataFrame 으로 불러온다 → 처음 5 행을 출력한다 → 각 열의 자료형과 결측치 정보를 확인한다 → 행·열의 크기와 열 이름을 확인한다.

확인하기 데이터는 394,560 행, 5 열이며 기준일자·기준시·본부명·전력사용량·고객호수 열로 구성되어 있음을 확인할 수 있습니다.

생각해 보기 데이터가 많을수록 바로 그래프를 그리기보다 먼저 head(), info(), shape, columns 로 구조를 확인하는 것이 중요합니다.

2. 기준일자를 날짜 형식으로 변환하기

의사코드 문자열 형태의 기준일자를 날짜 자료형으로 변환하여 새 열 '일자'를 만든다 → 변환 후 처음 5 행과 자료형을 다시 확인한다.

확인하기 '일자' 열이 새로 만들어지고 날짜를 다룰 수 있는 datetime 자료형으로 변환됩니다.

생각해 보기 날짜 자료형으로 바꾸어 두면 연도·월·일처럼 필요한 시간 정보를 쉽게 추출할 수 있습니다.

3. 날짜에서 월 정보만 추출하기

의사코드 '일자' 열에서 월 정보만 추출하여 새 열 '월'을 만든다 → 처음 5행을 출력하여 월 열이 추가되었는지 확인한다.

확인하기 각 행에 1~12의 월 정보가 추가되어 월별 분석을 위한 기준 열을 만들 수 있습니다.

생각해 보기 원본 데이터는 날짜·시간·본부별 여러 행으로 구성되어 있으므로 월별 비교를 하려면 먼저 월을 나타내는 열이 필요합니다.

4. 월별 전력사용량의 합계 구하기

의사코드 같은 월의 행들을 하나의 그룹으로 묶는다 → 각 월의 전력사용량을 모두 더한다 → index를 일반 열로 되돌려 월별 집계 DataFrame을 만든다 → 결과를 확인한다.

확인하기 1월부터 12월까지 한 달에 하나씩 총 12개의 전력사용량 합계가 만들어집니다.

생각해 보기 `groupby('월')`은 같은 월의 데이터를 묶고, 전력사용량의 `sum()`은 각 그룹의 사용량을 모두 더합니다.

5. 월별 전력사용량을 선그래프로 나타내기

의사코드 그래프 크기를 정한다 → 월을 x 축, 월별 전력사용량 합계를 y 축으로 선그래프를 그린다 → 그래프 창을 출력한다.

확인하기 1월부터 12월까지 전력사용량의 월별 변화가 시간 순서대로 선으로 연결되어 나타납니다.

생각해 보기 월은 시간의 순서가 있는 자료이므로 각 달의 변화 흐름을 확인할 때 선그래프가 적절합니다.

6. y 축의 범위를 조정하여 월별 차이 비교하기

의사코드 y 축의 보이는 범위를 100,000,000 에서 145,000,000 사이로 제한한다 → 그래프를 다시 출력하여 월별 차이를 비교한다.

확인하기 월별 차이가 더 뚜렷하게 보이며, 제공된 데이터에서는 8 월의 전력사용량이 가장 높고 4 월이 가장 낮게 나타납니다.

생각해 보기 ylim()으로 축의 범위를 좁히면 차이를 자세히 볼 수 있지만, 변화가 실제보다 크게 느껴질 수 있으므로 축의 범위를 함께 확인하며 해석해야 합니다.

전기차 충전소 지역별 분포 분석

1. CSV 파일을 불러오고 데이터 구조 확인하기

의사코드 전기차 충전소 CSV 파일을 cp949 인코딩으로 불러온다 → 처음 5 행, 데이터 정보, 행·열의 크기, 열 이름을 차례로 확인한다.

확인하기 제공된 데이터는 4,673 행, 5 열이며 충전소 아이디·충전소명·시도·위도·경도 열로 구성되어 있습니다.

생각해 보기 한글이 포함된 CSV는 저장 방식에 따라 인코딩을 지정해야 할 수 있습니다. 파일이 제대로 읽히지 않으면 인코딩을 먼저 확인합니다.

2. 주소에서 행정구역 이름만 추출하기

의사코드 시도 열의 문자열을 공백 기준으로 나눈다 → 첫 번째 단어만 가져와 '행정구역' 열을 만든다 → 행정구역·위도·경도 열만 남긴다.

확인하기 '서울특별시 중구 ...'와 같은 긴 주소에서 '서울특별시'처럼 첫 행정구역 이름만 분리되어 새 열에 저장됩니다.

생각해 보기 문자열 전체를 그대로 분석하기보다 분석 목적에 필요한 부분만 분리하면 지역별 집계가 쉬워집니다.

3. 행정구역 이름의 혼용 확인하기

의사코드 행정구역 열에 어떤 값들이 들어 있는지 중복을 제외하고 확인한다 → 같은 지역이 서로 다른 이름으로 기록되어 있는지 찾는다.

확인하기 표준화 전에는 '서울특별시'와 '서울', '경기도'와 '경기'처럼 같은 지역이 서로 다른 값으로 존재하며 총 30 개의 고유 값이 나타납니다.

생각해 보기 같은 지역을 다른 이름으로 기록한 상태에서 바로 개수를 세면 실제 한 지역이 여러 지역으로 나뉘어 집계될 수 있습니다.

4. 행정구역 이름을 표준화하기

의사코드 혼용된 지역명을 하나의 표준명으로 바꾸기 위한 딕셔너리를 만든다 → 행정구역 열의 값을 표준명으로 교체한다 → 다시 고유 값을 확인한다.

확인하기 '서울'은 '서울특별시', '경기'는 '경기도'처럼 통일됩니다. 제공된 코드의 기준으로 광주·전남 관련 표기도 '전남광주특별시'로 통합되며, 표준화 후 고유 행정구역은 16 개로 정리됩니다.

생각해 보기 전처리에서 이름을 통일하는 작업은 단순한 글자 수정이 아니라 같은 대상을 같은 범주로 묶기 위한 중요한 과정입니다.

5. 지역별 충전소 개수 집계하기

의사코드 표준화된 행정구역별 데이터 개수를 센다 → 지역별 개수를 저장한다 → 서로 다른 행정구역의 개수도 확인한다.

확인하기 지역별 충전소 개수가 큰 순서로 계산되며, 제공된 데이터에서는 경기도가 851 개, 서울특별시가 582 개로 집계됩니다. 표준화된 행정구역 수는 16 개입니다.

생각해 보기 `value_counts()`는 범주별 빈도를 빠르게 확인할 때 유용합니다. 전처리 전후 결과가 달라지는지도 비교해 봅니다.

6. 지역별 충전소 개수를 가로 막대그래프로 나타내기

의사코드 한글 글꼴을 설정한다 → 그래프 크기를 정한다 → 행정구역을 y 축, 지역별 충전소 개수를 x 축으로 하는 가로 막대그래프를 그린다 → 그래프 창을 출력한다.

확인하기 지역별 충전소 개수를 막대 길이로 비교할 수 있습니다. 지역명이 길기 때문에 세로 막대그래프보다 가로 막대그래프에서 이름을 읽기 쉽습니다.

생각해 보기 그래프를 그리기 전에 지역명 표준화를 하지 않았다면 같은 지역이 여러 막대로 나뉘어 나타날 수 있습니다. 시각화보다 전처리가 먼저입니다.

서울 모기지수 데이터 분석

1. CSV 파일을 불러오고 데이터 구조 확인하기

의사코드 'seoul_mosquito.csv'를 cp949 인코딩으로 DataFrame에 불러온다 → 처음 5행을 출력한다 → 각 열의 자료형과 결측치 정보를 확인한다 → 행·열의 크기를 확인한다.

확인하기 원본 데이터는 1,966행, 4열이며 모기지수 발생일을 포함한 각 열의 자료형과 결측치 여부를 확인할 수 있습니다.

생각해 보기 분석을 시작하기 전에 head(), info(), shape로 데이터의 구조를 먼저 확인하면 이후 전처리 기준을 세우기 쉽습니다.

2. 결측치와 중복 데이터 확인하기

의사코드 각 열의 결측치 개수를 확인한다 → 완전히 같은 행이 몇 개 중복되어 있는지 확인한다 → 중복으로 판단된 행의 index를 리스트 형태로 저장하고 출력한다.

확인하기 원본 1,966행에서 중복 행이 35개 존재하는 것을 확인할 수 있으며, 중복된 행의 index 목록도 확인할 수 있습니다.

생각해 보기 duplicated()는 기본적으로 처음 나타난 행은 남기고 이후에 다시 나타난 같은 행을 중복으로 판단합니다.

3. 중복 행을 제거하고 데이터 크기 다시 확인하기

의사코드 중복 행을 제거한다 → 제거된 DataFrame 의 행·열 크기를 다시 출력하여 전처리 전후 행 수를 비교한다.

확인하기 중복 행을 제거하면 데이터 크기가 (1966, 4)에서 (1931, 4)로 줄어듭니다.

생각해 보기 전처리를 한 뒤 shape 를 다시 확인하면 실제로 몇 개의 행이 제거되었는지 쉽게 검증할 수 있습니다.

4. 모기지수 발생일을 날짜 자료형으로 변환하기

의사코드 '모기지수 발생일' 열을 datetime 자료형으로 변환한다 → 처음 5 행을 출력한다 → info()로 자료형이 바뀌었는지 확인한다.

확인하기 문자열 형태의 날짜가 datetime 자료형으로 바뀌어 연도·월·일 등의 날짜 정보를 직접 추출할 수 있게 됩니다.

생각해 보기 날짜를 문자열 그대로 두면 연도 조건이나 월·일 추출을 하기 불편하므로 시계열 분석에서는 날짜형 변환이 중요합니다.

5. 전처리 전 모기지수 변화를 그래프로 확인하기

의사코드 모기지수 발생일을 x 축, 수변부 모기지수를 y 축으로 하는 선그래프를 그린다 → 비정상적으로 큰 값이 있는지 전체 흐름을 관찰한다.

확인하기 원본 그래프에서는 일부 값이 100 을 크게 넘어 전체 변화 흐름을 읽기 어렵게 만드는 구간을 발견할 수 있습니다.

생각해 보기 시각화는 결과를 보여주는 도구이면서 동시에 이상치를 발견하는 도구가 될 수 있습니다. 그래프가 이상하면 원자료를 다시 확인합니다.

6. 2020 년 5 월 이전의 100 초과 값 찾기

의사코드 발생일이 2020-05-01 보다 이전이면서 수변부 모기지수가 100 보다 큰 행의 index 를 찾는다 → 조건에 해당하는 행의 개수와 앞쪽 index 를 확인한다 → 예시 index 1249 의 값을 확인한다.

확인하기 조건에 해당하는 행들을 찾을 수 있고, 예시 index 1249 의 수변부 모기지수는 179.9 로 확인됩니다.

생각해 보기 여러 조건을 동시에 사용할 때는 각 조건을 괄호로 묶고 & 연산자로 연결합니다. 이상치를 바로 수정하기 전에 먼저 위치와 값을 확인합니다.

7. 이전 기간의 이상치를 10 으로 나누어 보정하기

의사코드 앞에서 찾은 index 의 수변부 모기지수 값을 선택한다 → 해당 값들을 10 으로 나누어 다시 저장한다 → 예시 index 1249 의 수정 전후 값을 비교한다.

확인하기 예시 index 1249 의 값은 179.9 에서 17.99 로 보정됩니다.

생각해 보기 loc 를 사용하면 특정 행과 특정 열을 함께 지정하여 값을 확인하거나 수정할 수 있습니다. 조건에 해당하는 행만 바뀌는지 확인합니다.

8. 2020 년의 100 초과 값을 상한 100 으로 보정하기

의사코드 발생 연도가 2020 년이면서 수변부 모기지수가 100 보다 큰 행의 index 를 찾는다 → 예시 값을 확인한다 → 조건에 해당하는 값은 모두 100 으로 바꾼다.

확인하기 예시 index 1084 의 값은 102.1 에서 100.0 으로 바뀝니다.

생각해 보기 같은 '100 초과'라도 기간에 따라 처리 규칙이 다를 수 있습니다. 데이터의 의미와 생성 방식에 맞는 전처리 기준이 필요합니다.

9. 날짜에서 월-일을 만들고 같은 날짜의 평균 구하기

의사코드 발생일에서 월-일만 추출하여 새 열 '월일'을 만든다 → 같은 월일끼리 묶는다 → 수변부 모기지수의 평균을 계산하고 index 를 일반 열로 되돌린다.

확인하기 여러 해에 걸쳐 같은 월·일에 관측된 모기지수가 하나의 평균값으로 정리되어 계절적 변화 패턴을 비교할 수 있습니다.

생각해 보기 연도를 제외하고 월-일만 묶으면 해마다 반복되는 계절 패턴을 살펴볼 수 있습니다. 분석 목적에 따라 그룹 기준이 달라집니다.

10. 월일별 평균 모기지수를 선그래프로 나타내기

의사코드 월일을 x 축, 월일별 평균 수변부 모기지수를 y 축으로 선그래프를 그린다 → x 축 눈금은 10 개 간격으로 표시한다 → 글자를 35 도 회전하여 읽기 쉽게 만든다 → 그래프를 출력한다.

확인하기 월일에 따른 평균 모기지수의 계절적 변화가 선으로 나타나며, x 축 글자를 일정 간격으로 표시해 날짜가 겹치는 문제를 줄일 수 있습니다.

생각해 보기 xticks()에서 눈금 위치와 표시할 글자를 함께 지정하면 많은 날짜를 모두 표시하지 않고 필요한 간격으로 줄여서 보여줄 수 있습니다.